

```
>>> from nltk.corpus import BracketParseCorpusReader
>>> corpus_root = r"C:\corpora\penntreebank\parsed\mrg\wsj" ❶
>>> file_pattern = r"*/wsj_.*\mrg" ❷
>>> ptb = BracketParseCorpusReader(corpus_root, file_pattern)
>>> ptb.fileids()
['00/wsj_0001.mrg', '00/wsj_0002.mrg', '00/wsj_0003.mrg', '00/wsj_0004.mrg', ...]
>>> len(ptb.sents())
49208
>>> ptb.sents(fileids='20/wsj_2013.mrg')[19]
['The', '55-year-old', 'Mr.', 'Noriega', 'is', "n't", 'as', 'smooth', 'as', 'the',
'shah', 'of', 'Iran', 'as', 'well-born', 'as', 'Nicaragua', "s", 'Anastasio',
'Somoza', 'as', 'imperial', 'as', 'Ferdinand', 'Marcos', 'of', 'the', 'Philippines',
'or', 'as', 'bloody', 'as', 'Haiti', "s", 'Baby', 'Doc', 'Duvalier', '.']
```

2 Conditional Frequency Distributions

We introduced frequency distributions in 3. We saw that given some list `mylist` of words or other items, `FreqDist(mylist)` would compute the number of occurrences of each item in the list. Here we will generalize this idea.

When the texts of a corpus are divided into several categories, by genre, topic, author, etc, we can maintain separate frequency distributions for each category. This will allow us to study systematic differences between the categories. In the previous section we achieved this using NLTK's `ConditionalFreqDist` data type. A **conditional frequency distribution** is a collection of frequency distributions, each one for a different "condition". The condition will often be the category of the text. 2.1 depicts a fragment of a conditional frequency distribution having just two conditions, one for news text and one for romance text.

Condition: News		Condition: Romance	
the		the	
cute		cute	
Monday		Monday	
could		could	
will		will	

Figure 2.1: Counting Words Appearing in a Text Collection (a conditional frequency distribution)

2.1 Conditions and Events

A frequency distribution counts observable events, such as the appearance of words in a text. A conditional frequency distribution needs to pair each event with a condition. So instead of processing a sequence of words ❶, we have to process a sequence of pairs ❷:

```
>>> text = ['The', 'Fulton', 'County', 'Grand', 'Jury', 'said', ...] ❶
>>> pairs = [('news', 'The'), ('news', 'Fulton'), ('news', 'County'), ...] ❷
```

Each pair has the form (condition, event). If we were processing the entire Brown Corpus by genre there would be 15 conditions (one per genre), and 1,161,192 events (one per word).

2.2 Counting Words by Genre

In 1 we saw a conditional frequency distribution where the condition was the section of the Brown Corpus, and for each condition we counted words. Whereas `FreqDist()` takes a simple list as input, `ConditionalFreqDist()` takes a list of pairs.

```
>>> from nltk.corpus import brown
>>> cfd = nltk.ConditionalFreqDist(
...     (genre, word)
...     for genre in brown.categories()
...     for word in brown.words(categories=genre))
```

Let's break this down, and look at just two genres, news and romance. For each genre ❷, we loop over every word in the genre ❸, producing pairs consisting of the genre and the word ❶:

```
>>> genre_word = [(genre, word) ❶
...               for genre in ['news', 'romance'] ❷
...               for word in brown.words(categories=genre)] ❸
>>> len(genre_word)
170576
```

So, as we can see below, pairs at the beginning of the list `genre_word` will be of the form `('news', word)` ❶, while those at the end will be of the form `('romance', word)` ❷.

```
>>> genre_word[:4]
[('news', 'The'), ('news', 'Fulton'), ('news', 'County'), ('news', 'Grand')] # [_start-
genre]
>>> genre_word[-4:]
[('romance', 'afraid'), ('romance', 'not'), ('romance', ''), ('romance', '.')] # [_end-
genre]
```

We can now use this list of pairs to create a `ConditionalFreqDist`, and save it in a variable `cfd`. As usual, we can type the name of the variable to inspect it ❶, and verify it has two conditions ❷:

```
>>> cfd = nltk.ConditionalFreqDist(genre_word)
>>> cfd ❶
<ConditionalFreqDist with 2 conditions>
>>> cfd.conditions()
['news', 'romance'] # [_conditions-cfd]
```

Let's access the two conditions, and satisfy ourselves that each is just a frequency distribution:

```
>>> print(cfd['news'])
<FreqDist with 14394 samples and 100554 outcomes>
>>> print(cfd['romance'])
<FreqDist with 8452 samples and 70022 outcomes>
>>> cfd['romance'].most_common(20)
[(',', 3899), (',', 3736), ('the', 2758), ('and', 1776), ('to', 1502),
('a', 1335), ('of', 1186), ('"', 1045), ('"', 1044), ('was', 993),
('I', 951), ('in', 875), ('he', 702), ('had', 692), ('?', 690),
('her', 651), ('that', 583), ('it', 573), ('his', 559), ('she', 496)]
>>> cfd['romance']['could']
193
```

2.3 Plotting and Tabulating Distributions

Apart from combining two or more frequency distributions, and being easy to initialize, a `ConditionalFreqDist` provides some useful methods for tabulation and plotting.

The plot in 1.1 was based on a conditional frequency distribution reproduced in the code below. The condition is either of the words *america* or *citizen* ❷, and the counts being plotted are the number of times the word occurred in a particular speech. It exploits the fact that the filename for each speech, e.g., `1865-Lincoln.txt` contains the year as the first four characters ❶. This code generates the pair `('america', '1865')` for every instance of a word whose lowercased form starts with *america* — such as *Americans* — in the file `1865-Lincoln.txt`.

```
>>> from nltk.corpus import inaugural
>>> cfd = nltk.ConditionalFreqDist(
...     (target, fileid[:4]) ❶
...     for fileid in inaugural.fileids()
...     for w in inaugural.words(fileid)
...     for target in ['america', 'citizen'] ❷
...     if w.lower().startswith(target))
```

The plot in 1.2 was also based on a conditional frequency distribution, reproduced below. This time, the condition is the name of the language and the counts being plotted are derived from word lengths ❶. It exploits the fact that the filename for each language is the language name followed by `-Latin1` (the character encoding).

```
>>> from nltk.corpus import udhr
>>> languages = ['Chickasaw', 'English', 'German_Deutsch',
...             'Greenlandic_Inuktitut', 'Hungarian_Magyar', 'Ibibio_Efik']
>>> cfd = nltk.ConditionalFreqDist(
...     (lang, len(word)) ❶
...     for lang in languages
...     for word in udhr.words(lang + '-Latin1'))
```

In the `plot()` and `tabulate()` methods, we can optionally specify which conditions to display with a `conditions=` parameter. When we omit it, we get all the conditions. Similarly, we can limit the samples to display with a `samples=` parameter. This makes it possible to load a large quantity of data into a conditional frequency distribution, and then to explore it by plotting or tabulating selected conditions and samples. It also gives us full control over the order of conditions and samples in any displays. For example, we can tabulate the cumulative frequency data just for two languages, and for words less than 10 characters long, as shown below. We interpret the last cell on the top row to mean that 1,638 words of the English text have 9 or fewer letters.

```
>>> cfd.tabulate(conditions=[ 'English', 'German_Deutsch' ],
...               samples=range(10), cumulative=True)
           0    1    2    3    4    5    6    7    8    9
English    0  185  525  883  997 1166 1283 1440 1558 1638
German_Deutsch 0  171  263  614  717  894 1013 1110 1213 1275
```

Note

Your Turn: Working with the news and romance genres from the Brown Corpus, find out which days of the week are most newsworthy, and which are most romantic. Define a variable called `days` containing a list of days of the week, i.e. ['Monday', ...]. Now tabulate the counts for these words using `cfd.tabulate(samples=days)`. Now try the same thing using `plot` in place of `tabulate`. You may control the output order of days with the help of an extra parameter: `samples=[ 'Monday', ... ]`.

You may have noticed that the multi-line expressions we have been using with conditional frequency distributions look like list comprehensions, but without the brackets. In general, when we use a list comprehension as a parameter to a function, like `set([w.lower() for w in t])`, we are permitted to omit the square brackets and just write: `set(w.lower() for w in t)`. (See the discussion of "generator expressions" in [4.2](#) for more about this.)

2.4 Generating Random Text with Bigrams

We can use a conditional frequency distribution to create a table of bigrams (word pairs). (We introduced bigrams in [3](#).) The `bigrams()` function takes a list of words and builds a list of consecutive word pairs. Remember that, in order to see the result and not a cryptic "generator object", we need to use the `list()` function:

```
>>> sent = [ 'In', 'the', 'beginning', 'God', 'created', 'the', 'heaven',
...          'and', 'the', 'earth', '.' ]
>>> list(nltk.bigrams(sent))
[('In', 'the'), ('the', 'beginning'), ('beginning', 'God'), ('God', 'created'),
 ('created', 'the'), ('the', 'heaven'), ('heaven', 'and'), ('and', 'the'),
 ('the', 'earth'), ('earth', '.')]

```

In [2.2](#), we treat each word as a condition, and for each one we effectively create a frequency distribution over the following words. The function `generate_model()` contains a simple loop to generate text. When we call the function, we choose a word (such as `'living'`) as our initial context, then once inside the loop, we print the current value of the variable `word`, and reset `word` to be the most likely token in that context (using `max()`); next time through the loop, we use that word as our new context. As you can see by inspecting the output, this simple approach to text generation tends to get stuck in loops; another method would be to randomly choose the next word from among the available words.

```
def generate_model(cfdist, word, num=15):
    for i in range(num):
        print(word, end=' ')
        word = cfdist[word].max()

text = nltk.corpus.genesis.words('english-kjv.txt')
bigrams = nltk.bigrams(text)
cfd = nltk.ConditionalFreqDist(bigrams) ❶

>>> cfd['living']
FreqDist({'creature': 7, 'thing': 4, 'substance': 2, ',': 1, '.': 1, 'soul': 1})
>>> generate_model(cfd, 'living')
living creature that he said , and the land of the land of the land

```

Example 2.2 (code `random_text.py`): Figure 2.2: Generating Random Text: this program obtains all bigrams from the text of the book of Genesis, then constructs a conditional frequency distribution to record which words are most likely to follow a given word; e.g., after the word *living*, the most likely word is *creature*; the `generate_model()` function uses this data, and a seed word, to generate random text.

Conditional frequency distributions are a useful data structure for many NLP tasks. Their commonly-used methods are summarized in [2.1](#).

Table 2.1:

NLTK's Conditional Frequency Distributions: commonly-used methods and idioms for defining, accessing, and visualizing a conditional frequency distribution of counters.

Example	Description
<code>cfdist = ConditionalFreqDist(pairs)</code>	create a conditional frequency distribution from a list of pairs
<code>cfdist.conditions()</code>	the conditions
<code>cfdist[condition]</code>	the frequency distribution for this condition
<code>cfdist[condition][sample]</code>	frequency for the given sample for this condition
<code>cfdist.tabulate()</code>	tabulate the conditional frequency distribution

Example	Description
<code>cfdist.tabulate(samples, conditions)</code>	tabulation limited to the specified samples and conditions
<code>cfdist.plot()</code>	graphical plot of the conditional frequency distribution
<code>cfdist.plot(samples, conditions)</code>	graphical plot limited to the specified samples and conditions
<code>cfdist1 < cfdist2</code>	test if samples in <code>cfdist1</code> occur less frequently than in <code>cfdist2</code>

3 More Python: Reusing Code

By this time you've probably typed and retyped a lot of code in the Python interactive interpreter. If you mess up when retyping a complex example you have to enter it again. Using the arrow keys to access and modify previous commands is helpful but only goes so far. In this section we see two important ways to reuse code: text editors and Python functions.

3.1 Creating Programs with a Text Editor

The Python interactive interpreter performs your instructions as soon as you type them. Often, it is better to compose a multi-line program using a text editor, then ask Python to run the whole program at once. Using IDLE, you can do this by going to the File menu and opening a new window. Try this now, and enter the following one-line program:

```
print('Monty Python')
```

Save this program in a file called `monty.py`, then go to the Run menu, and select the command `Run Module`. (We'll learn what modules are shortly.) The result in the main IDLE window should look like this:

```
>>> ===== RESTART =====
>>>
Monty Python
>>>
```

You can also type `from monty import *` and it will do the same thing.

From now on, you have a choice of using the interactive interpreter or a text editor to create your programs. It is often convenient to test your ideas using the interpreter, revising a line of code until it does what you expect. Once you're ready, you can paste the code (minus any `>>>` or `...` prompts) into the text editor, continue to expand it, and finally save the program in a file so that you don't have to type it in again later. Give the file a short but descriptive name, using all lowercase letters and separating words with underscore, and using the `.py` filename extension, e.g., `monty_python.py`.

Note

Important: Our inline code examples include the `>>>` and `...` prompts as if we are interacting directly with the interpreter. As they get more complicated, you should instead type them into the editor, without the prompts, and run them from the editor as shown above. When we provide longer programs in this book, we will leave out the prompts to remind you to type them into a file rather than using the interpreter. You can see this already in [2.2](#) above. Note that it still includes a couple of lines with the Python prompt; this is the interactive part of the task where you inspect some data and invoke a function. Remember that all code samples like [2.2](#) are downloadable from <http://nltk.org/>.

3.2 Functions

Suppose that you work on analyzing text that involves different forms of the same word, and that part of your program needs to work out the plural form of a given singular noun. Suppose it needs to do this work in two places, once when it is processing some texts, and again when it is processing user input.

Rather than repeating the same code several times over, it is more efficient and reliable to localize this work inside a **function**. A function is just a named block of code that performs some well-defined task, as we saw in [1](#). A function is usually defined to take some inputs, using special variables known as **parameters**, and it may produce a result, also known as a **return value**. We define a function using the keyword `def` followed by the function name and any input parameters, followed by the body of the function. Here's the function we saw in [1](#) (including the `import` statement that is needed for Python 2, in order to make division behave as expected):

```
>>> from __future__ import division
>>> def lexical_diversity(text):
...     return len(text) / len(set(text))
```

We use the keyword `return` to indicate the value that is produced as output by the function. In the above example, all the work of the function is done in the `return` statement. Here's an equivalent definition which does the same work using multiple lines of code. We'll change the parameter name from `text` to `my_text_data` to remind you that this is an arbitrary choice:

```
>>> def lexical_diversity(my_text_data):
...     word_count = len(my_text_data)
...     vocab_size = len(set(my_text_data))
...     diversity_score = vocab_size / word_count
...     return diversity_score
```

Notice that we've created some new variables inside the body of the function. These are **local variables** and are not accessible outside the function. So now we have defined a function with the name `lexical_diversity`. But just defining it won't produce any output! Functions do nothing until they are "called" (or "invoked"):

```
>>> from nltk.corpus import genesis
>>> kjv = genesis.words('english-kjv.txt')
>>> lexical_diversity(kjv)
0.06230453042623537
```

Let's return to our earlier scenario, and actually define a simple function to work out English plurals. The function `plural()` in [3.1](#) takes a singular noun and generates a plural form, though it is not always correct. (We'll discuss functions at greater length in [4.4](#).)

```
def plural(word):
    if word.endswith('y'):
        return word[:-1] + 'ies'
    elif word[-1] in 'sx' or word[-2:] in ['sh', 'ch']:
        return word + 'es'
    elif word.endswith('an'):
        return word[:-2] + 'en'
    else:
        return word + 's'

>>> plural('fairy')
'fairies'
>>> plural('woman')
'women'
```

Example 3.1 (code `plural.py`): Figure 3.1: A Python Function: this function tries to work out the plural form of any English noun; the keyword `def` (define) is followed by the function name, then a parameter inside parentheses, and a colon; the body of the function is the indented block of code; it tries to recognize patterns within the word and process the word accordingly; e.g., if the word ends with `y`, delete the `y` and add `ies`.

The `endswith()` function is always associated with a string object (e.g., `word` in [3.1](#)). To call such functions, we give the name of the object, a period, and then the name of the function. These functions are usually known as **methods**.

3.3 Modules

Over time you will find that you create a variety of useful little text processing functions, and you end up copying them from old programs to new ones. Which file contains the latest version of the function you want to use? It makes life a lot easier if you can collect your work into a single place, and access previously defined functions without making copies.

To do this, save your function(s) in a file called (say) `text_proc.py`. Now, you can access your work simply by importing it from the file:

```
>>> from text_proc import plural
>>> plural('wish')
wishes
>>> plural('fan')
fen
```

Our plural function obviously has an error, since the plural of *fan* is *fans*. Instead of typing in a new version of the function, we can simply edit the existing one. Thus, at every stage, there is only one version of our plural function, and no confusion about which one is being used.

A collection of variable and function definitions in a file is called a Python **module**. A collection of related modules is called a **package**. NLTK's code for processing the Brown Corpus is an example of a module, and its collection of code for processing all the different corpora is an example of a package. NLTK itself is a set of packages, sometimes called a **library**.

Caution!

If you are creating a file to contain some of your Python code, do *not* name your file `nltk.py`: it may get imported in place of the "real" NLTK package. When it imports modules, Python first looks in the current directory (folder).